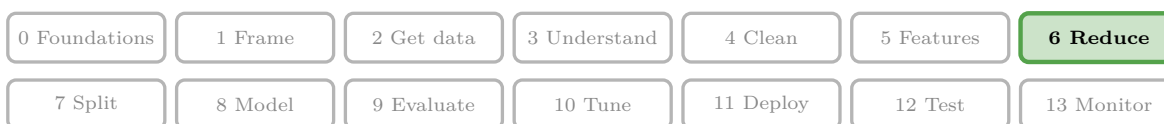


# PCA in Practice: MNIST

## Contents

|                                                     |    |
|-----------------------------------------------------|----|
| 1. Overview . . . . .                               | 2  |
| 2. The MNIST data . . . . .                         | 2  |
| 3. KNN on all 784 features . . . . .                | 3  |
| 4. KNN on principal components . . . . .            | 3  |
| 4.1 The steps . . . . .                             | 3  |
| 4.2 Accuracy as k grows . . . . .                   | 4  |
| 4.3 What standardising would do . . . . .           | 6  |
| 5. Visualising the digits . . . . .                 | 6  |
| 5.1 In 2D . . . . .                                 | 6  |
| 5.2 In 3D . . . . .                                 | 7  |
| 6. What a fitted PCA stores . . . . .               | 8  |
| 7. How many components to keep . . . . .            | 9  |
| 7.1 Explained variance . . . . .                    | 9  |
| 7.2 The cumulative curve and the 90% rule . . . . . | 9  |
| 8. When PCA does not help . . . . .                 | 11 |
| 9. Summary . . . . .                                | 12 |
| 10. Sources . . . . .                               | 12 |
| 11. Key terms . . . . .                             | 12 |

**Where this fits:** Pipeline map step 6 (Reduce dimensions). See the [Course map](#).



- **Builds on:** Unsupervised learning ([Note 3](#)); Standardization ([Note 24](#)); Feature scaling ([Note 24](#)); Curse of dimensionality ([Note 46](#)); Feature extraction ([Note 47](#)); Variance ([Note 47](#)).
- **Compare with:** Low-rank approximation (truncated SVD) ([Note 612](#)).

## 1. Overview

**Key point:** On real image data, PCA cuts 784 features to 50 and KNN gets slightly *more* accurate and much faster. PCA also lets us see the data in 2D and 3D.

A **feature** is an input variable (one column of the data table), an **observation** is one record (one row), and the **target** is the output we predict. Here each observation is one image, each feature one pixel, and the target the digit.

The previous two Notes built PCA from the geometry and the mathematics. This Note uses scikit-learn's PCA on a real dataset of handwritten digits, for PCA's two main jobs:

1. **Fewer features:** train a model on a few principal components instead of every pixel, and compare accuracy and speed (Sections 3 and 4).
2. **Visualisation:** squeeze the data into 2 or 3 features and plot it (Section 5).

Then we answer two practical questions. How many components should we keep (Section 7)? And when does PCA not help at all (Section 8)?

## 2. The MNIST data

**Key point:** Each image is  $28 \times 28$  pixels, stored as one row of 784 columns plus a label from 0 to 9.

**MNIST** is a well-known set of images of handwritten digits, 0 to 9. Each image is  $28 \times 28 = 784$  grey pixels, and each pixel's brightness runs from 0 (white) to 255 (black).

To put images in a table, each image becomes one row with one column per pixel: the first 28 columns are the top row of the image, the next 28 the second row, and so on. One more column holds the label, the digit the image shows. Reshaping a row back into  $28 \times 28$  shows the picture again.

We use 42,000 images and split them 80/20: 33,600 for training and 8,400 for testing. The task is classification: from the 784 pixel values, predict the digit.

**Extra:** The images here come from the full MNIST set on OpenML (70,000 images), loaded with `fetch_openml("mnist_784")`. We take a random 42,000, the same size as the Digit Recognizer competition file on Kaggle, which is also drawn from MNIST and

needs a Kaggle account to download.

**Python:** Loading the data and splitting it.

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

X, y = fetch_openml("mnist_784", version=1,
                    return_X_y=True, as_frame=False)
# (a random 42,000 rows are kept; see the Notebook)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)
```

### 3. KNN on all 784 features

**Key point:** KNN reaches 96.8% on the raw pixels, but each prediction compares the image with 33,600 others across 784 features, which is slow.

We first train **KNN** (K-nearest neighbours) with its default of 5 neighbours on all 784 features. To classify a test image, KNN computes its distance to every one of the 33,600 training images and takes the most common label among the 5 nearest.

The result: **96.8% accuracy**, but each of the 8,400 test images needs 33,600 distances in 784 dimensions. On an idle machine the run took about 19 seconds. The heavy arithmetic is the computation problem of the curse of dimensionality (Note 46).

### 4. KNN on principal components

**Key point:** Fit PCA on the training set, transform both sets, then train the same KNN on the new features.

#### 4.1 The steps

**Key point:** PCA learns its components from the training set only, like a scaler, and then transforms both sets.

PCA in scikit-learn follows the same fit and transform pattern as scalers and encoders:

1. **Fit** PCA on the training set: it centres the pixels, computes the covariance matrix and its eigenvectors.
2. **Transform** both sets: project every image onto the kept components.
3. **Train KNN** on the transformed training set and test it on the transformed test set.

We do not standardise the pixels first. All 784 pixels share one unit, brightness from 0 to 255, and when features are measured in the same units there is no need to scale them before PCA (ISL §12.2.4). Section 4.3 shows what standardising would cost here.

The parameter `n_components` sets how many components to keep. Left as `None`, it keeps all of them, 784 here.

```
Python: KNN on 50 principal components.  
from sklearn.decomposition import PCA  
from sklearn.neighbors import KNeighborsClassifier  
  
pca = PCA(n_components=50).fit(X_train)      # train only  
Z_train = pca.transform(X_train)           # (33600, 50)  
Z_test = pca.transform(X_test)             # (8400, 50)  
  
knn = KNeighborsClassifier(n_neighbors=5).fit(Z_train, y_train)  
knn.score(Z_test, y_test)                  # 0.973
```

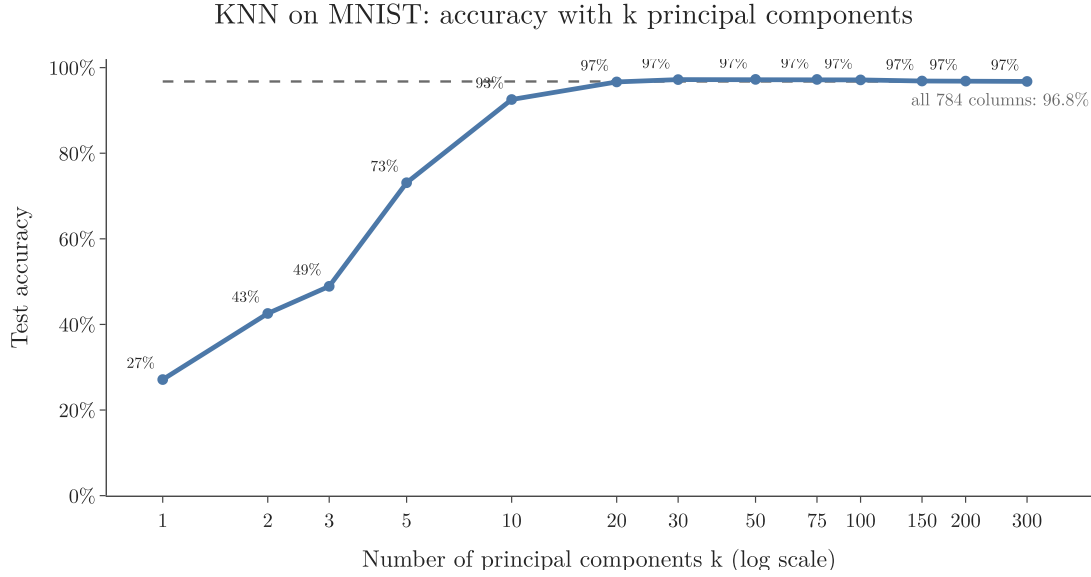
With 50 components the accuracy is **97.3%**, slightly above the 96.8% on all 784 pixels. Each distance now uses 50 numbers instead of 784, about 16 times less arithmetic; on an idle machine the run took about 1 second. We dropped 734 of the 784 features and lost nothing. Like a summary of a long report, the 50 components keep what matters and leave out the repetition: neighbouring pixels mostly move together, so a few combinations describe them well.

**Extra:** Timings depend on the machine and on what else runs on it. On a busy shared machine the same two runs took 120 to 200 seconds and 1.5 to 3 seconds. The gap stays more than tenfold.

## 4.2 Accuracy as $k$ grows

**Key point:** Accuracy climbs steeply over the first 20 components, peaks around 30 to 75, and then slowly drifts back to the all-pixel level.

How many components do we need? Figure 1 repeats the experiment for  $k$  from 1 to 300 components.



**Figure 1:** KNN accuracy on MNIST with k principal components; the dashed line is KNN on all 784 pixels

| k        | 1   | 2   | 3   | 5   | 10  | 20    | 50    | 100   | 300   |
|----------|-----|-----|-----|-----|-----|-------|-------|-------|-------|
| Accuracy | 27% | 43% | 49% | 73% | 93% | 96.7% | 97.3% | 97.1% | 96.8% |

- **1 to 20 components:** each one adds a lot; the first components carry the most information.
- **30 to 75:** the best range, 97.2% to 97.3%, above the 96.8% of all 784 pixels.
- **Above 100:** accuracy drifts back down towards 96.8%. The later components add small differences between images that do not help KNN find the right neighbours (see the Extra below).

So 50 components, 6% of the original features, give the best result in this run.

**Extra:** Are the later components useful? Components 51 to 784 alone give 54%, so they hold a little information. Next to the first 50, though, they act like noise for KNN: shuffling each of them across images (same spread, no link to the label) leaves accuracy unchanged, 96.8% real against 97.0% shuffled. Components 101 to 300 behave the same way (97.1% with shuffled ones). Dropping them removes small, unhelpful differences, which is why 50 components beat all 784 pixels. All results come from one split with 8,400 test images, so one image is 0.012 points.

### 4.3 What standardising would do

**Key point:** Standardising every pixel first lowers accuracy, to 94.0% on all pixels and 95.4% with 50 components.

Standardising (Note 24) gives every pixel mean 0 and standard deviation 1. For images that is harmful. A pixel near the edge is blank in almost every image; standardising stretches its rare small changes to the same size as the busy centre pixels, so they disturb the distances KNN measures.

| Setup                 | Accuracy     |
|-----------------------|--------------|
| Raw pixels, all 784   | 96.8%        |
| Raw pixels, PCA 50    | <b>97.3%</b> |
| Standardised, all 784 | 94.0%        |
| Standardised, PCA 50  | 95.4%        |

**Extra:** The rarely used pixels are the main culprit. Dropping the pixels that are non-zero in fewer than 5% of the training images, then standardising the rest, gives 95.9%, so most of the loss comes back (Notebook, last cell). Standardising remains the right step when features are in different units, such as age in years and income in rupees.

## 5. Visualising the digits

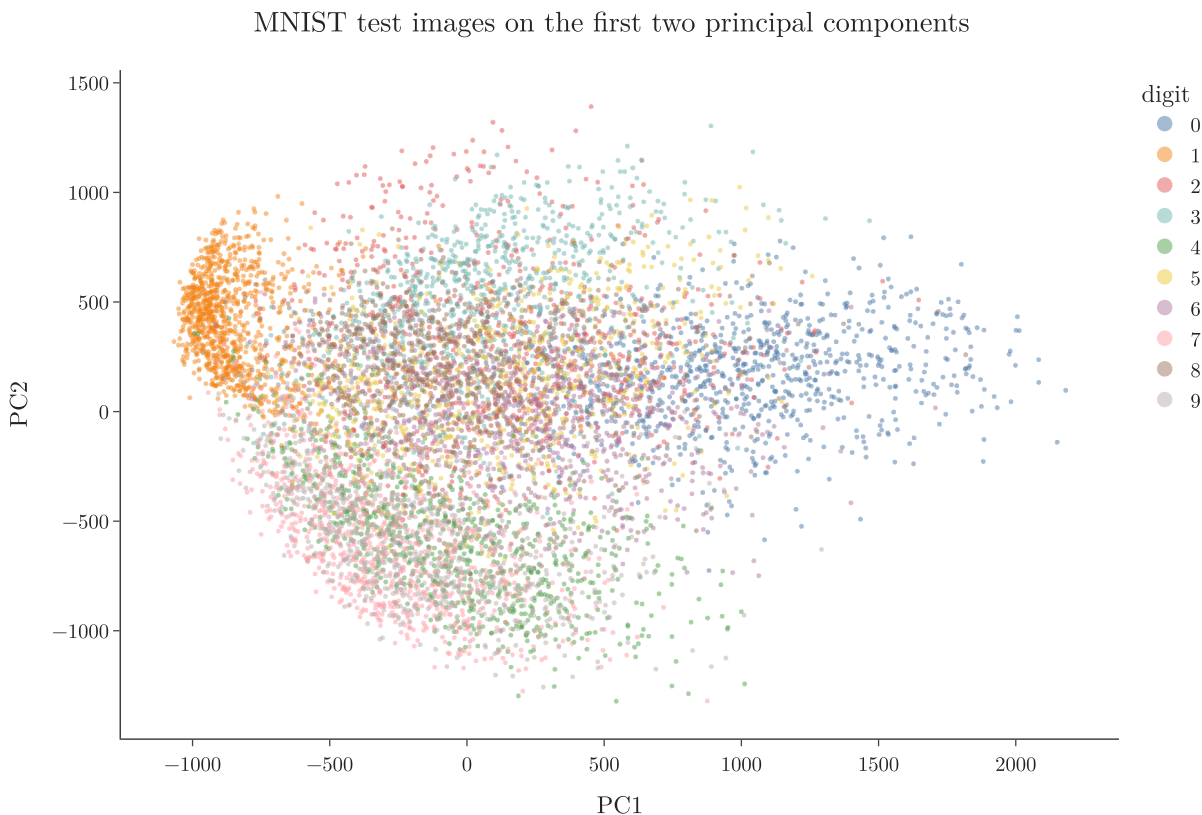
**Key point:** With 2 or 3 components we can draw all 784-dimensional images as points and see which digits sit apart and which overlap.

PCA's second job is visualisation. We cannot plot 784 dimensions, but we can plot the first 2 or 3 principal components.

### 5.1 In 2D

**Key point:** In 2D the digits overlap heavily, but some groups are already visible: 1s on one side, 0s on the other.

Figure 2 shows the 8,400 test images on PC1 and PC2, coloured by their label.



**Figure 2:** MNIST test images on PC1 and PC2

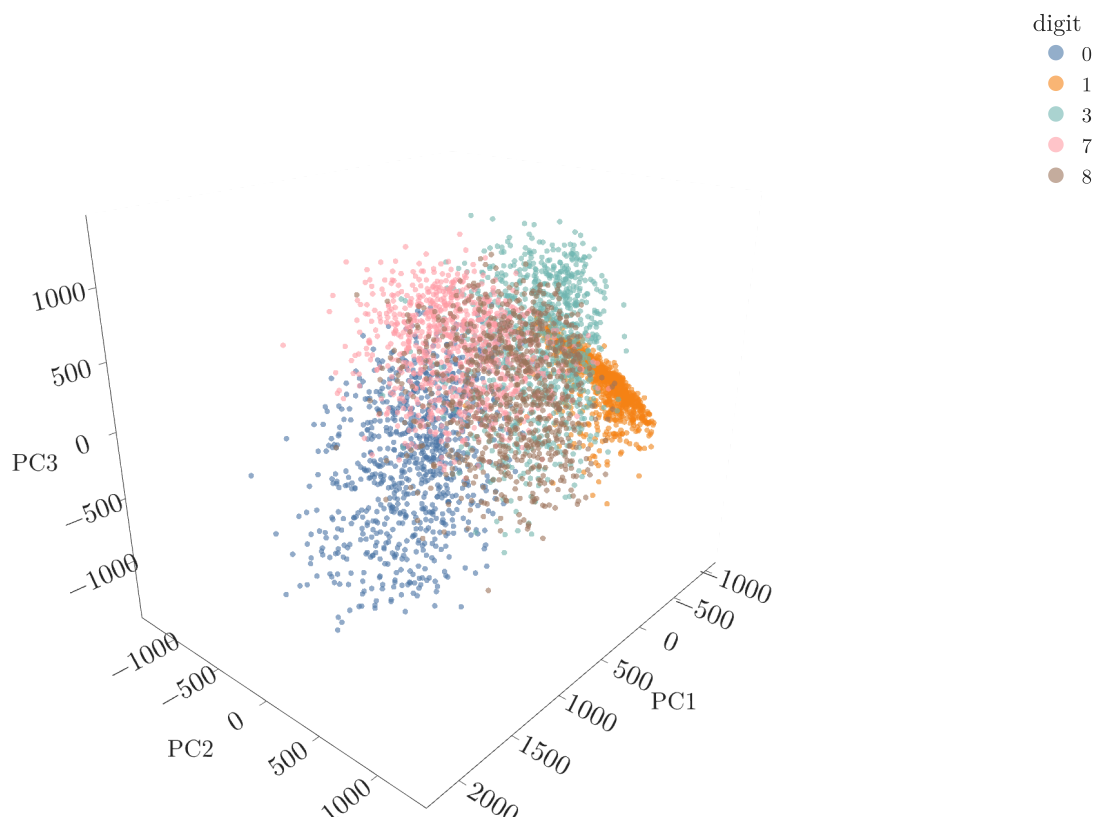
The colours overlap a lot. Still, a pattern shows. The 1s (orange) form a tight group on the left and the 0s (blue) spread out on the right. PC1 roughly measures how much ink a digit uses. A 1 is a single thin stroke (85 inked pixels on average); a 0 is a big loop (191). In the Notebook, PC1 and the number of inked pixels have a correlation of 0.86.

## 5.2 In 3D

**Key point:** A third component separates more digits. Similar-looking digits stay mixed.

Figure 3 adds PC3 and shows five digits only, to keep the picture readable.

Digits 0, 1, 3, 7 and 8 on the first three principal components



**Figure 3:** Digits 0, 1, 3, 7 and 8 on PC1, PC2 and PC3

- **0 and 1** sit at opposite ends of PC1: a big loop against a thin stroke.
- **3** rises along PC3, away from the others.
- **8** sits in the middle, mixed with 3 and 7.

In the Notebook this plot is interactive: we can turn it, zoom, and click digits in the legend to hide or show them. Turning it shows other close groups: 4, 7 and 9 share the same low PC2 values (average about  $-570$  each).

## 6. What a fitted PCA stores

**Key point:** `explained_variance_` holds the eigenvalues, `components_` holds the eigenvectors, and `explained_variance_ratio_` holds each component's share of the variance.

After `fit`, a `PCA` object keeps the results of its eigen-decomposition (from the previous Note):

| Attribute                        | What it holds                  | For 3 components on MNIST |
|----------------------------------|--------------------------------|---------------------------|
| <code>explained_variance_</code> | the eigenvalues, largest first | 331,121, 245,937, 211,045 |
| <code>components_</code>         | the eigenvectors, one per row  | shape (3, 784)            |

| Attribute                              | What it holds                             | For 3 components on MNIST |
|----------------------------------------|-------------------------------------------|---------------------------|
| <code>explained_variance_ratio_</code> | each eigenvalue divided by the sum of all | 9.7%, 7.2%, 6.2%          |

The eigenvalues are large because the pixels run from 0 to 255, so their variances are in the thousands. Each eigenvector has 784 numbers because it is a direction in the 784-dimensional pixel space. Reshaped to  $28 \times 28$ , each one is itself a picture: it shows which pixels that component combines.

The first three components together hold only 23% of the variance. So little variance explains why the 2D and 3D pictures overlap so much: they show only a small part of what makes the images different.

## 7. How many components to keep

**Key point:** Add up the explained variance ratios from the largest down, and keep enough components to reach about 90% of the variance.

### 7.1 Explained variance

**Key point:** Each eigenvalue, divided by the sum of all eigenvalues, is the share of the data's variance that its component explains.

Each eigenvalue  $\lambda_i$  is the variance along component  $i$ . The total variance of the data is the sum of all of them.

In words: the share of variance a component explains is its eigenvalue divided by the sum of all eigenvalues.

$$\text{explained variance ratio}_i = \frac{\lambda_i}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

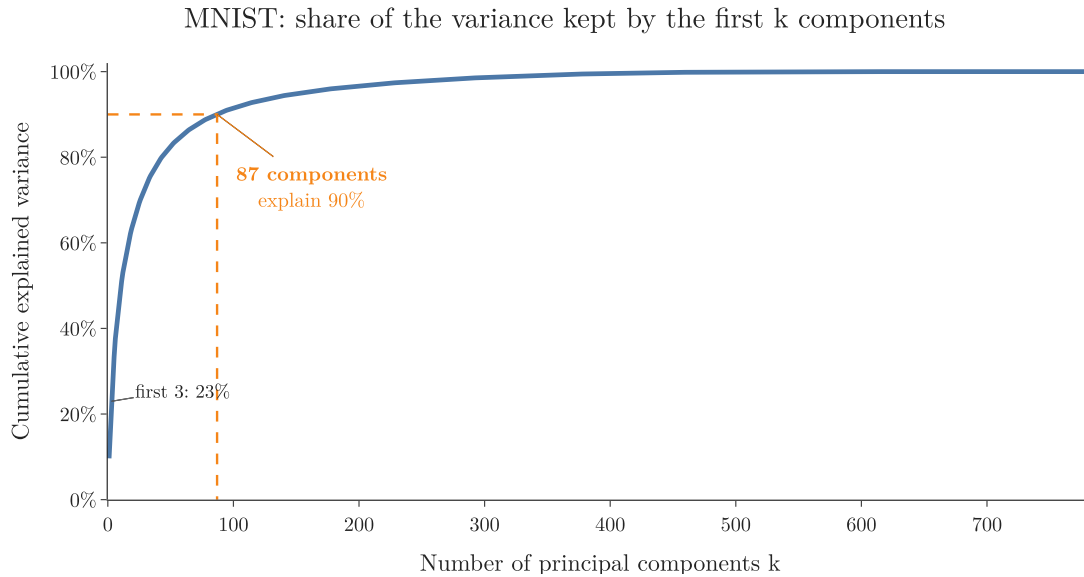
With numbers: on MNIST the 784 eigenvalues add up to about 3,429,600. PC1's eigenvalue is 331,121, so

$$\frac{331,121}{3,429,600} \approx 0.097 = 9.7\%$$

### 7.2 The cumulative curve and the 90% rule

**Key point:** On MNIST, 87 components explain 90% of the variance.

Adding up the ratios in order gives the **cumulative explained variance**: the share kept by the first  $k$  components. Figure 4 plots it for every  $k$ .



**Figure 4:** Cumulative explained variance of MNIST's components

The curve rises fast and then flattens towards 100%. A common rule of thumb is to keep enough components to explain about **90%** of the variance: on MNIST, **87 components**. For 95% it takes 153.

**Python:** Finding k for 90%.

```
import numpy as np

full = PCA().fit(X_train)           # keep all 784 components
cum = np.cumsum(full.explained_variance_ratio_)
np.searchsorted(cum, 0.90) + 1     # 87

# or let scikit-learn choose: a number between 0 and 1
# means "keep this share of the variance"
PCA(n_components=0.90).fit(X_train).n_components_ # 87
```

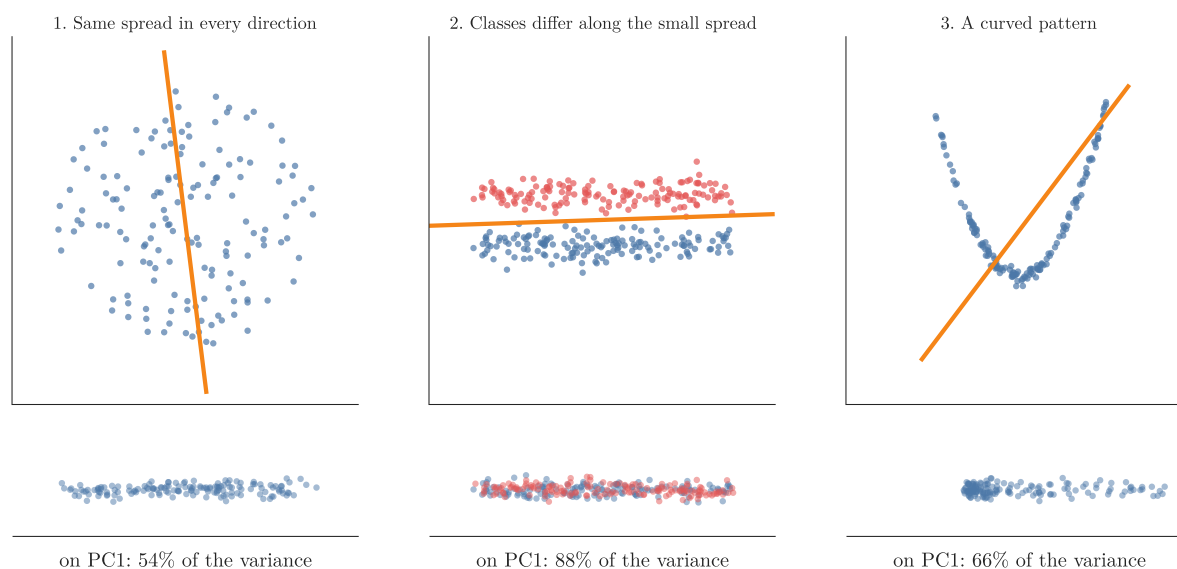
`np.cumsum` gives the running total. `np.searchsorted` finds the first position where it reaches 0.90; positions start at 0, so we add 1.

**Extra:** The 90% rule is a safe starting point, not a law. In Figure 1, KNN was already at its best with 50 components, which explain about 83% of the variance. When there is a model to evaluate, the most direct way to pick  $k$  is to try several values and compare the test or cross-validation scores, as Section 4.2 did.

## 8. When PCA does not help

**Key point:** PCA only finds straight directions of large spread. It fails when there is no such direction, when the useful difference lies along a small-spread direction, or when the pattern is curved.

PCA is powerful, but it can only rotate the axes and drop some. Figure 5 shows three kinds of data where rotating and dropping is not enough. In each, the orange line is PC1, and the strip below shows the points projected onto it.



**Figure 5:** Three cases where PCA does not help

1. **The same spread in every direction.** The points form a round cloud. Every direction has about the same variance, so PC1 holds only about half of it (54%). Whichever axis we drop, we lose as much as we keep.
2. **The classes differ along the small spread.** Two long, thin groups lie side by side. PC1 runs along their length, which has the most variance, and holds 88% of it. But the two classes differ across their width. Projected onto PC1, they fall on top of each other.
3. **A curved pattern.** The points follow a U shape. PCA can only project onto a straight line, so the U is flattened, and points from the two arms of the U land in the same place.

In all three cases, the picture in fewer dimensions mixes up points that were clearly apart, and a model trained on it does worse.

**Extra:** Case 2 shows that PCA is unsupervised: `fit` takes only  $\mathbf{X}$ , never the labels, so it keeps spread, not class differences. A supervised method, **LDA** (linear discriminant analysis, also called Fisher’s linear discriminant), looks for the direction that separates the classes instead (PRML §4.1.4). For curved patterns (case 3), non-linear methods exist, such as kernel PCA (PRML §12.3) and t-SNE (van der Maaten and Hinton 2008).

## 9. Summary

| Question                           | Answer on MNIST                                              |
|------------------------------------|--------------------------------------------------------------|
| Accuracy with all 784 raw pixels   | 96.8%, about 19 seconds (idle machine)                       |
| Accuracy with 50 components        | 97.3%, about 1 second                                        |
| Standardising the pixels first     | lowers accuracy (94.0% all pixels, 95.4% with 50 components) |
| Components for 90% of the variance | 87                                                           |
| Variance in the first 3 components | 23%                                                          |
| Digits at opposite ends of PC1     | 0 and 1                                                      |
| Digits that overlap                | 8 with 3 and 7; 4, 7 and 9                                   |

- Use PCA like a scaler: fit on the training set, transform both sets.
- Pixels share one unit, so PCA runs on them unscaled; standardising them hurts KNN here.
- A few principal components can keep, or even improve, the accuracy at a fraction of the computation.
- 2 or 3 components let us look at high-dimensional data, but they hold only part of its information.
- `explained_variance_ratio_` and its running total tell how much of the data each  $k$  keeps; about 90% is a common target.
- PCA fails when spread is equal in all directions, when classes differ along a small-spread direction, or when the pattern is curved.

## 10. Sources

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning* (PRML). Springer. §4.1.4 Fisher’s linear discriminant; §12.3 Kernel PCA.
- ISL: James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). *An Introduction to Statistical Learning*, 2nd ed. Springer. §12.2.4, More on PCA: scaling the variables.
- van der Maaten, L. and Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research* 9: 2579–2605.

## 11. Key terms

| Term        | Meaning                                                                      |
|-------------|------------------------------------------------------------------------------|
| Feature     | An input variable: one column of the data table; here one pixel              |
| Observation | One record: one row of the data table; here one image                        |
| Target      | The output we predict; here the digit                                        |
| MNIST       | A dataset of 70,000 images of handwritten digits, $28 \times 28$ pixels each |

| Term                          | Meaning                                                                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------|
| n_components                  | The number of principal components PCA keeps; a number between 0 and 1 means a share of the variance       |
| explained_variance__          | The eigenvalues of the fitted PCA, largest first                                                           |
| components__                  | The eigenvectors of the fitted PCA, one per row                                                            |
| Explained variance ratio      | One component's share of the total variance: its eigenvalue divided by the sum of all                      |
| Cumulative explained variance | The share of the variance kept by the first k components together                                          |
| LDA                           | Linear discriminant analysis: a supervised method that finds the directions that best separate the classes |